

EasyCrypt Proof-Script Commentary for the HAETAE Provable-Security Artifact

May 2026

1 Purpose

This note records manuscript-ready commentary for the EasyCrypt proof scripts in the HAETAE provable-security artifact. The goal is to state precisely what the checked development establishes, what remains conditional, and which proof obligations should be targeted next.

Verification command. The checked proof surface is the manifest `provable-security/proof-files.txt`. The repository verification gate is:

```
sh provable-security/verify-provable-security.sh
```

The gate compiles each listed EasyCrypt file with include paths `provable-security/easycrypt` and `kyber-security`. In the current run, all sixteen manifest files compiled successfully and the script ended with `RESULT: PASS`.

2 Honest Machine-Checked Claim

The current EasyCrypt development machine-checks a formal HAETAE model, perfect correctness for that model, and a family of conditional EUF-CMA reduction theorems in the random-oracle setting. The strongest checked EUF-CMA theorem bounds the adversary success probability by the counted HAETAE ROM bound, assuming an explicit exact-hyperball NMA bound and abstract MLWE and Module-SIS hardness bounds. The artifact does not prove unconditional cryptographic security, does not discharge the cryptographic hardness assumptions, and does not contain a HAETAE SUF-CMA theorem beyond the generic SUF-CMA game definition.

3 Claim Matrix

Claim	EasyCrypt item	Status
HAETAE signing and verification are perfectly correct in the EasyCrypt model.	<code>correctness_perfect</code> , <code>HAETAE_Security.ec:40</code>	Machine-checked, model-conditional.
The EUF-CMA experiment is defined.	<code>module EUF_CMA</code> , <code>Sig_ROM.ec:94</code>	Game/interface definition only.

Claim	EasyCrypt item	Status
The SUF-CMA experiment is defined.	<code>module SUF_CMA, Sig_ROM.ec:174</code>	Definition only. No HAETAE SUF-CMA theorem was found in the checked proof surface.
The abstract HAETAE EUF-CMA bound follows from a Fiat–Shamir hop, an NMA bound, a bimodal-to-MSIS hop, and MLWE/Module-SIS hardness bounds.	<code>euf_cma_security, HAETAE_Security.ec:136</code>	Conditional/abstract reduction.
The counted-ROM HAETAE EUF-CMA bound follows from the exact-hyperball NMA premise and abstract hardness premises.	<code>euf_cma_security_from_checked_rom_exact_hyperball_and_counted_bimodal, HAETAE_Security.ec:1217</code>	Machine-checked conditional reduction.
The budget-explicit path exposes the signing-budget adapters and the sampling-loss obligation.	<code>euf_cma_security_from_budgeted_sampling_loss_and_counted_bimodal, HAETAE_Security.ec:1170</code>	Conditional, exposes proof obligations.
The bimodal self-target MSIS game can be related to Module-SIS.	<code>bimodal_to_module_sis_reduction, HAETAE_Security.ec:418</code>	Checked wrapper, hardness-conditional.

4 Dependency Chain for the Strongest EUF-CMA Theorem

The strongest theorem in the current checked surface is `euf_cma_security_from_checked_rom_sampling_hop_and_counted_bimodal` in `HAETAE_Security.ec:1217`. Its conclusion is:

$$\text{Pr}[\text{SIG.EUF_CMA}(H, \text{HAETAE}, A).\text{main}() \text{ @ } \&m : \text{res}] \leq \text{haetae_euf_counted_rom_bound}.$$

The theorem depends on the following chain.

1. The theorem applies `euf_cma_security_from_rom_exact_hyperball_sampling_hop_and_counted_bimodal` (`HAETAE_Security.ec:1036`).
2. The real-signing to RO exact-hyperball hop is supplied by `real_signing_rom_exact_hyperball_hop_checked` (`HAETAE_Security.ec:1102`).
3. That lemma applies `real_signing_rom_exact_hyperball_hop_from_rom_signing_attempt_hop` (`HAETAE_Security.ec:865`) and then calls `rom_internal_nma_rom_signing_attempt_rom_exact_hyperball_loss_bound` (`HAETAE_HopGames.ec:7554`).
4. The current proof of `rom_internal_nma_rom_signing_attempt_rom_exact_hyperball_loss_bound` is a coarse fallback: it uses probability boundedness and `rejection_sampling_loss_term_ge1` rather than an adversary-level sampling hybrid. This makes the statement checked, but not a non-vacuous cryptographic sampling-loss proof.

5. The final EUF-CMA theorem also assumes the exact-hyperball NMA premise:

$$\Pr[\text{UF_NMA}(\dots \text{ROExactHyperballPaperSimSampler } \dots)] + \text{rejection_sampling_loss_term} \leq \Pr[\text{MLWE_Game}(\text{Bmlwe})] + \Pr[\text{BimodalSelfTargetMSIS_Game}(\text{Bbimodal})].$$

This premise is not discharged by the theorem itself.

6. The MLWE and Module-SIS terms are abstract hardness premises: `mlwe_hardness_term` and `module_sis_hardness_term` in `HAETAE_Reductions.ec:9-10`.

5 Trust Boundaries

Abstract types and distributions. The model declares `mlwe_instance` as abstract and introduces lossless distributions such as `dmlwe_real`, `dmlwe_random`, `dmodule_sis_instance`, and `dbimodal_selftarget_msis_instance` (`HAETAE_Assumptions.ec:9-19`). These are part of the formal model boundary.

Hardness terms. The quantities `mlwe_hardness_term` and `module_sis_hardness_term` are abstract real-valued bounds (`HAETAE_Reductions.ec:9-10`). EasyCrypt checks reductions to these terms; it does not establish the concrete hardness of MLWE or Module-SIS.

Declared modules. The theorem sections declare adversaries and reductions, for example `H`, `A`, `Bmlwe`, `Bbimodal`, and `Bsis` in `HAETAE_Security.ec:129-134`. The checked statements are parametric over these interfaces.

Coarse bounds. Several lemmas use the fact that a probability is at most one together with `rejection_sampling_loss_term_ge1`. The important remaining coarse sites are `HAETAE_HopGames.ec:7305`, `HAETAE_HopGames.ec:7402`, `HAETAE_HopGames.ec:7554`, and `HAETAE_HopGames.ec:9593`. These are checked EasyCrypt facts, but they should be described as fallback or vacuous bounds, not as tight cryptographic reductions.

6 Recent Proof-Script Improvement

The latest proof improvement extends the bridge chain in `provable-security/easycrypt/HAETAE_HopGames.ec`. The first is `concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_state_loss` at line 6844. It lifts the existing clean one-call sampler loss to predicates that may mention the final random-oracle global state, provided the proof supplies left and right framing inequalities. The second is `concrete_budgeted_o_sign_sample_with_seed_clean_state_loss_surface` at line 6912, specialized to the budgeted NMA adapter’s ROM query logs. The latest addition is `concrete_budgeted_o_sign_clean_stateful_loss_from_sample_frames` at line 8034. It composes the state-aware sampler bridge with explicit attempt-side and exact-side `0.sign` frame inequalities, proving that one clean active signing call can be lifted with one `rejection_sampling_loss_term` once the continuation has already been summarized as predicates over the returned signature, the sampled `paper_sim_signature_sample`, and the final random-oracle state.

These lemmas are non-vacuous because they reuse the checked one-call loss instead of relying on `rejection_sampling_loss_term_ge1`. They are still bridge or surface lemmas: they do not yet replace the adversary-level NMA fallback at `HAETAE_HopGames.ec:7554`, and the concrete clean NMA lifting at `HAETAE_HopGames.ec:9593` remains coarse.

Current first-active two-call work. The proof script now contains a seed-sensitive suffix attempt for the actual first-active two-call `0.sign` branch. The new EasyCrypt surface introduces `concrete\fro\get\_step`, a pure model of the checked lazy random-oracle `FR0.get` map update, and `concrete\_two\_call\_first\_active\_o\_sign\_suffix\_kernel`, which records the sampler-expand query, reconstructs the sampler ROM entry from the sampled signing coins, runs the message-hash and challenge-hash suffix, computes the signature/transcript, and evaluates the full-state postcondition. The seeded bridge `structural\_to\_exact\_hyperball\_seeded\_coin\_sa` keeps the stronger point-loss premise and does not use the coarse NMA target. Verification is pending in this edit pass; the commentary should only be upgraded to a checked claim after `HAETAE\_HopGames.ec` and the full manifest compile successfully.

7 Publication Wording

Acceptable wording. We machine-checked in EasyCrypt a HAETAE random-oracle model, a perfect correctness theorem, and conditional EUF-CMA reduction chains with explicit MLWE and Module-SIS hardness assumptions and stated sampling/ROM trust boundaries.

Too-strong wording to avoid. The manuscript should not say that HAETAE is fully machine-checked secure, unconditionally secure, SUF-CMA verified, or that every cryptographic reduction and sampling hop has been discharged non-vacuously.

Precise statement. The EasyCrypt artifact verifies a conditional EUF-CMA theorem for the modeled HAETAE signature scheme: assuming the stated exact-hyperball NMA bound and abstract MLWE/Module-SIS hardness bounds, the adversary’s EUF-CMA success probability is bounded by the counted HAETAE ROM security expression. The proof is machine-checked over the listed EasyCrypt files, but the final cryptographic claim remains conditional on the abstract hardness assumptions, declared reduction interfaces, and several explicit sampling/ROM boundaries, including coarse fallback bounds that should not be presented as tight reductions.

8 Recommended Next Codex Request

The highest-value next request is to target the smallest adversary-level gap that the new bridge lemmas make approachable. The request should also make the manuscript file a live work product: every proof-status change must be reflected in this document before the agent stops.

You are a Codex AI agent in the current HAETAE EasyCrypt project. Task: replace the coarse EasyCrypt proof of the budgeted clean NMA sampling lifting with a non-vacuous proof using the checked one-call sampling-loss bridges. Start from `concrete_rom_internal_budgeted_nma_ro_sig` in `provable-security/easycrypt/HAETAE_HopGames.ec`. Do not make broad refactors. Preserve existing theorem statements unless a minimal strengthening is necessary. Use `concrete_budgeted_o_sign_sample_with_seed_clean_state_loss_surface` and related projection lemmas to prove the clean conditioned NMA bound by lifting one `0.sign` call and accumulating at most `rom_signature_query_budget * rejection_sampling_loss_term`. Avoid using `rejection_sampling_loss_term_ge1` or probability-boundedness as the main proof.

Continuously maintain the manuscript commentary while working. Update `manuscript/haetae-easycrypt-proof` whenever a proof status changes: before editing, record the target and current trust boundary; after each successful proof step, update the relevant claim matrix, dependency chain, and recent proof-script improvement text; after any failed attempt, record the blocker only if it changes the proof plan or trust boundary; after verification, update the commands/results and publication wording if the honest claim changed. Keep the LaTeX concise and manuscript-ready, not as a raw work log.

Rerun `sh provable-security/verify-provable-security.sh` and compile the manuscript LaTeX if `pdflatex` is available. Report exactly which coarse-bound dependency was removed, which premises remain, which manuscript sections were updated, and any new proof obligations.

9 Continuous Documentation Protocol

Proof development and manuscript maintenance should proceed together. The document is not merely an after-the-fact summary: it is the running record of the current honest security claim. A Codex agent working on the proof should update the manuscript at four checkpoints:

1. **Before proof edits:** identify the target lemma, the current fallback or premise it relies on, and the expected non-vacuous replacement.
2. **After a checked proof step:** update the claim matrix and dependency chain if a lemma changes status from coarse, premise-level, or abstract to non-vacuously proved.
3. **After a failed proof attempt:** update the proof plan only when the failure reveals a real missing lemma, interface mismatch, or trust boundary. Do not turn transient tactic failures into manuscript content.
4. **After verification:** record the exact EasyCrypt verification command, the result, the changed theorem status, and any change to acceptable publication wording.

Current Proof-Edit Checkpoint

The active proof target is `concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball` in `provable-security/easycrypt/HAETAE_HopGames.ec`. Its current proof is a coarse fallback: it bounds the clean conditioned NMA event by one and then uses `rom_signature_query_budget * rejection_sampling_loss_term >= 1`. The intended replacement is a non-vacuous lifting argument that applies the checked one-call bridge `concrete_budgeted_o_sign_sample_with_seed_clean_state` inside the budgeted signing oracle and accumulates at most `rom_signature_query_budget * rejection_sampling_loss_term` over the checked signing-query budget.

Proof Attempt Status

A new checked local support lemma, `concrete_budgeted_o_sign_clean_stateful_loss_from_sample_frames` now proves the one-call stateful frame step. Its proof is non-vacuous: it uses `concrete_budgeted_o_sign_sample` and does not appeal to `rejection_sampling_loss_term_ge1` or probability boundedness.

The direct replacement of the target NMA lemma is still blocked by an adversary-continuation gap rather than by a local sampling lemma. The new stateful frame lemma handles one active `0.sign` call after the pre-call state and post-call continuation have already been summarized by explicit frame inequalities. The target NMA lemma needs a stronger rule: after an active signing call, the postcondition is the rest of the adaptive adversary execution plus the final UF-NMA verifier. That continuation depends on `glob A`, `glob HAETAE_RO.FRO`, the transcript/signing logs, the query counters, and the clean flag, not only on the returned signature.

Consequently, replacing the target proof non-vacuously requires one of the following checked ingredients before the target lemma can be closed:

1. a stateful approximate call rule for `ROMInternalTranscriptBudgetedPaperSimAsNMA(...).0.sign` that accepts continuation predicates over the full post-call state and adds one `rejection_sampling_loss_term` or

2. a concrete hybrid wrapper, analogous to EasyCrypt’s hybrid theorem, that packages adversary hash queries as leaks, packages signing as the hybridized oracle, proves the one-switch stateful signing lemma, and sums the loss over `signature_query_budget_count` active signing calls.

The current target lemma therefore remains in the trust boundary until that adversary-level stateful lifting principle is proved. Replacing its body with a proof that merely routes through another coarse lemma, or again uses `rejection_sampling_loss_term_ge1`, would not change the honest security claim.

The current follow-up isolates and checks the saturated half of that missing interface. The proof script now contains `concrete_budgeted_o_sign_saturated_fallback_equiv` (HAETAE_HopGames.ec:867), `concrete_budgeted_o_sign_saturated_fallback_full_state_equiv` (HAETAE_HopGames.ec:8701), `concrete_budgeted_ah_get_full_state_equiv` (HAETAE_HopGames.ec:8758), `concrete_budgeted_adversary_forge_saturated_clean_fallback_lifts` (HAETAE_HopGames.ec:8832), and `concrete_budgeted_adversary_forge_saturated_clean_fallback_lifts` (HAETAE_HopGames.ec:8920). Together these lemmas show that, once `signature_query_budget_count` $\leq$ `signing_count`, the adaptive `A.forge` continuation can keep making hash and signing calls without incurring additional sampler loss: hash calls are simulated identically, and post-budget signing calls take the same fallback branch in the signing-attempt and exact-hyperball worlds while preserving the ROM, transcript, query-log, and clean-flag state. The new probability-lifting wrapper converts the saturated equivalence into the exact inequality needed for predicates of the form `R res (glob HAETAE_RO.FRO)` under the clean event.

This is a checked adversary-level saturated-continuation fact, but it is not yet the active-call accumulation theorem. The remaining interface is the one-switch active-call hybrid: before saturation, choose one active call satisfying `signing_count < signature_query_budget_count`, apply `concrete_budgeted_o_sign_clean_stateful_loss_from_sample_frames` (HAETAE_HopGames.ec:8034) to that call, carry the full ROM and transcript state through the remaining `A.forge` continuation, then sum exactly one `rejection_sampling_loss_term` for each active call. No unchecked one-switch lemma was kept in the source.

The current active-call follow-up adds an explicit checked two-call hybrid wrapper for the concrete route: `ROMInternalTranscriptBudgetedTwoCallHybridAsNMA` (HAETAE_HopGames.ec:1766), plus the three intended instances `ROMInternalTranscriptBudgetedTwoCallG0AsNMA` (HAETAE_HopGames.ec:1875), `ROMInternalTranscriptBudgetedTwoCallG1AsNMA` (HAETAE_HopGames.ec:1885), and `ROMInternalTranscriptBudgetedTwoCallG2AsNMA` (HAETAE_HopGames.ec:1891). The wrapper keeps the same ROM, transcript, query-log, and clean-flag state as the budgeted game, but selects `First.sample_with_seed` on the first active call and `Second.sample_with_seed` on the second active call. Thus `G0` uses signing-attempt sampling twice, `G1` switches the first active call to exact-hyperball sampling, and `G2` uses exact-hyperball sampling twice. These are now real EasyCrypt games, not just comments or planned notation.

The same follow-up retains two checked counter facts for the concrete two-call route: `budgeted_paper_sim_sign_oracle_first_active_saturates` (HAETAE_HopGames.ec:1926) and `budgeted_paper_sim_sign_oracle_second_active_saturates` (HAETAE_HopGames.ec:1943). These lemmas use `signature_query_budget_count = 2` to show that the first active `O.sign` call moves `signing_count` from 0 to 1, and the second active call moves it from 1 to the saturated budget value. They justify reducing the intended hybrid proof to two active switches followed by the already checked saturated continuation, but they do not by themselves compose the sampling loss through the adaptive continuation.

The current first-switch follow-up proves the exact continuation needed after the first active signing call, but does not yet prove the approximate `G0 -> G1` bound itself. Once the shared two-call state satisfies `1 <= signing_count`, `G0` and `G1` no longer use the different first-call sampler: the next active call uses the signing-attempt `Second` sampler on both sides, and post-budget calls use the same fallback branch. This is checked at the hash-oracle level by `concrete_two_call_g0_g1_post_first_ah_get_equiv` (HAETAE_HopGames.ec:9002), at the signing-oracle level by `concrete_two_call_g0_g1_post_first_o_sign_equiv` (HAETAE_HopGames.ec:9080), and through the adaptive adversary continuation by `concrete_two_call_g0_g1_post_first_adversary_forge`

(HAETAE_HopGames.ec:9171). The probability wrapper `concrete_two_call_g0_g1_post_first_clean_adversary_lifts` (HAETAE_HopGames.ec:9264) packages this as a clean-event continuation fact with zero additional sampler loss. The remaining missing principle was the approximate split at the first active `0.sign` call: apply the one-call rejection-sampling loss to that call and then resume with the checked post-first continuation.

The latest first-active-call follow-up adds the checked frame-composition lemma `concrete_two_call_g0_g1_first_active_o_sign_clean_full_state_loss_from_c` (HAETAE_HopGames.ec:8509). This is the two-call-wrapper analogue of the one-call stateful bridge: under clean ROM coverage, equal sampler secret-key state, and `signing_count = 0`, it turns explicit attempt-side and exact-side sample-frame premises into a one `rejection_sampling_loss_term` bound from `ROMInternalTranscriptBudgetedTwoCallG0AsNMA.0.sign` to `ROMInternalTranscriptBudgetedTwoCallG1.0.sign`. Its predicate surface includes the returned signature, the final ROM global, and the full two-call adapter state (`pk_current`, `queries`, `transcripts`, `records`, `adversary_hash_count`, `signing_count`, `adversary_hash_queries`, `sampler_expand_queries`, and `sampler_bad_prequery`), and the event records `1 <= signing_count`. This makes the lemma continuation-ready, but the general full-state sample-frame inequalities remain premises of this generic lemma.

The newest concrete frame step discharges the signature-level first-active `0.sign` switch without using `mu_bounded`, `rejection_sampling_loss_term_ge1`, or the coarse target lemma. The attempt side is checked by `concrete_two_call_g0_first_active_o_sign_active_clean_signature_structural_to_exact_hyperball_coin_sample_pair_suffix_loss_from_point_loss` (HAETAE_HopGames.ec:8236) and `concrete_two_call_g0_first_active_o_sign_to_self_logged_sample_pair_suffix_loss_from_point_loss` (HAETAE_HopGames.ec:8386). The exact side is checked by `concrete_two_call_g1_first_active_o_sign_active_clean_signature_structural_to_exact_hyperball_coin_sample_pair_suffix_loss_from_point_loss` (HAETAE_HopGames.ec:8319) and `concrete_two_call_g1_exact_sample_with_seed_to_first_active_o_sign_active_clean_signature_structural_to_exact_hyperball_coin_sample_pair_suffix_loss_from_point_loss` (HAETAE_HopGames.ec:8466). These projections are then packaged in `concrete_two_call_g0_g1_first_active_o_sign_clean_full_state_loss_from_c` (HAETAE_HopGames.ec:8614), which gives a concrete `G0.0.sign <= G1.0.sign + rejection_sampling_loss_term` bound for signature predicates under `signing_count = 0`, clean ROM coverage, and equal sampler secret-key state. The remaining adversary-level gap is to lift that first-call switch to predicates over the full post-call ROM/transcript/two-call adapter state and then apply `concrete_two_call_g0_g1_post_first_clean_adversary_lifts` as the adaptive continuation.

The latest full-state-lift pass checked the seed-sensitive bridge that was missing from the signature-only switch. The helper `concrete_mu_dlet_ge_point` (HAETAE_HopGames.ec:8672) proves that one point contribution is bounded by the probability of the whole continuation bind. Using that helper, `structural_to_exact_hyperball_coin_sample_pair_suffix_loss_from_point_loss` (HAETAE_HopGames.ec:8683) lifts the rejection-sampling point-loss obligation through an arbitrary common randomized suffix over `signing_attempt_coin_sample_pair`. This bridge is seed-sensitive: the suffix may observe the sampled signing seed as well as the sampled paper simulation value.

The checked wrapper `concrete_two_call_g0_g1_first_active_o_sign_clean_full_state_loss_from_c` (HAETAE_HopGames.ec:8743) packages that bridge for the desired full-state `G0.0.sign <= G1.0.sign + rejection_sampling_loss_term` surface. Its final predicate ranges over the returned signature, `glob` `HAETAE\_RO.FRO`, `pk_current`, query/transcript/record logs, counters, adversary and sampler query logs, and `sampler_bad_prequery`. The lemma is still a frame-composition result: it requires concrete left and right frame inequalities connecting the actual two-call `0.sign` code to a common suffix kernel `K`. Those frame inequalities are now the exact remaining obligation for the full-state first-call switch.

This new bridge deliberately uses the stronger `structural_to_exact_hyperball_sample_pair_point_loss` not only the paper-sample predicate obligation. The stronger premise is needed because the full post-call state can depend on the seed recorded in `sampler_expand_queries`, and because the two post-sampler `FRO.get` calls make the final ROM global a randomized continuation of the coin/sample pair rather than a boolean predicate on `paper_sim_signature_sample`. No new unchecked EasyCrypt assumption was added. The concrete full-state `0.sign` switch, the adversary-level `G0 -> G1` lift, and the coarse NMA target lemma remain unchanged.

The exact endpoint of the two-call hybrid is now connected to the existing exact-hyperball

budgeted game by checked equivalences. The proof script first aligns the hash oracle in `concrete_two_call_g2_e` (`HAETAH_HopGames.ec:9312`), then aligns the signing oracle in `concrete_two_call_g2_exact_budgeted_o_sign` (`HAETAH_HopGames.ec:9389`). The signing-oracle proof explicitly case-splits on the active/saturated branch and on whether the active call is the first or second one, forcing the wrapper’s `First/Second` dispatch before simulation. These two oracle equivalences are lifted through `A.forge` by `concrete_two_call_g2` (`HAETAH_HopGames.ec:9462`), through the wrapper’s public `forge` procedure by `concrete_two_call_g2_exact` (`HAETAH_HopGames.ec:9554`), and finally through the NMA game by `concrete_two_call_g2_exact_budgeted` (`HAETAH_HopGames.ec:9569`). This closes the exact G2 endpoint; the remaining work is the adversary-level `G0 -> G1` lift, now starting from a checked signature-level first-call switch, and the second active-call switch `G1 -> G2`.

Verification Checkpoint

After the current follow-up, the EasyCrypt source retains the checked stateful one-call frame lemma, the checked first-active two-call sample-frame composition lemma, the checked concrete first-active signature-level `G0/G1 0.sign` loss lemma, the checked explicit two-call hybrid games, the checked post-first `G0/G1` continuation, the checked exact G2 endpoint equivalence, the checked active counter lemmas, the checked saturated adversary-continuation equivalence, and the checked saturated clean probability-lifting wrapper. The edited EasyCrypt file was locally checked with:

```
easycrypt compile provable-security/easycrypt/HAETAH_HopGames.ec -max-provers
1 -I provable-security/easycrypt -I kyber-security
```

The file compiled successfully. The full proof gate was rerun with:

```
sh provable-security/verify-provable-security.sh
```

The command compiled all sixteen manifest files successfully and ended with `RESULT: PASS`. The target clean NMA lifting theorem `concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hy` (`HAETAH_HopGames.ec:9593`) remains machine-checked only as a coarse fallback; the checked one-call stateful frame lemma, explicit two-call hybrid games, first-active two-call sample-frame composition lemma, concrete first-active signature-level `0.sign` switch, post-first `G0/G1` continuation, exact G2 endpoint equivalence, active counter lemmas, and saturated adversary-continuation lemmas are non-vacuous pieces, but they have not yet removed that fallback. This manuscript file was also recompiled with `pdflatex -interaction=nonstopmode -halt-on-error`; compilation succeeded, with only overfull-box warnings from long EasyCrypt identifiers.

Seed-Sensitive Suffix-Frame Follow-Up

The current seed-sensitive suffix-frame follow-up has not yet produced a checked full-state first-active `0.sign` switch. The active proof attempt instantiates the seeded suffix-kernel bridge shape for the actual two-call first-active branch, but the EasyCrypt compile currently stops in `concrete\_two\_call\_g0\_first\_active\_o\_sign\_clean\_full\_state\_frame\_to\_seeded\_coin`. The remaining checked obligation is local: the post-sampler two `FR0.get` conditionals must be shown to imply the rejected clean signature event from the first-active precondition, without expanding into a coarse `mu\_bounded` or `rejection\_sampling\_loss\_term\_ge1` argument.

The most recent compile command was:

```
timeout 600s easycrypt compile provable-security/easycrypt/HAETAH\_HopGames.ec
-max-provers 1 -I provable-security/easycrypt -I kyber-security
```

It failed at the `G0` frame proof with `cannot prove goal (strict)` after trying to discharge the branch with `mu\_ro\_output\_implies\_rejected\_state\_le0`. Therefore the full-state

first-active $G_0 \rightarrow G_1$ $O.\text{sign}$ switch and the adversary-level lift through `concrete\_two\_call\_g0\_g1\_post` remain unchecked. The stronger `structural\_to\_exact\_hyperball\_sample\_pair\_point\_loss\_oblig` premise is still present, and the coarse NMA target lemma has not been changed.

10 Prioritized Proof Plan

1. Replace `concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_condi` (`HAETAE_HopGames.ec:9593`). Expected edits: `provable-security/easycrypt/HAETAE_HopGames.ec`. Difficulty: high. Strategy: prove an adversary-call hybrid over the budgeted $O.\text{sign}$ oracle, apply `concrete_budgeted_o_sign_clean_stateful_loss_from_sample_frames` at the selected clean active signing call, use `budgeted_paper_sim_sign_oracle_first_active_sets_one` and `budgeted_paper_sim_sign_oracle_second_active_saturates` to split the two active switches, use `concrete_budgeted_adversary_forge_saturated_clean_fallback_lifts` for the post-budget continuation, and accumulate the loss by the checked signing-count budget.
2. Replace `rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound` (`HAETAE_HopGames.ec`). Expected edits: `provable-security/easycrypt/HAETAE_HopGames.ec` and possibly `HAETAE_Security.ec` if the final theorem should use the budget-explicit path. Difficulty: medium-high. Strategy: route through budgeted clean lifting and the bad-prequery finite-event-loss bound rather than through `rejection_sampling_loss_term_ge1`.
3. Discharge the exact-hyperball NMA-to-hardness premise in `euf_cma_security_from_checked_ro_sampling` (`HAETAE_Security.ec:1217`). Expected edits: `HAETAE_HopGames.ec`, `HAETAE_Transcript.ec`, `HAETAE_Assumptions.ec`, and `HAETAE_Security.ec`. Difficulty: high. Strategy: connect the exact-hyperball NMA game to the MLWE and bimodal self-target MSIS games via the existing transcript, special-soundness, and bimodal-to-Module-SIS machinery.